

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Case Study

1. A software company is using MD5 to store user passwords in its database. After a security breach, it is discovered that attackers were able to crack multiple passwords. Analyze the weaknesses of MD5 in this scenario and propose a more secure hashing approach using SHA-based techniques.
2. A banking application implements digital signatures for transaction verification, but users report unauthorized transactions being approved. Analyze the possible flaws in the digital signature implementation and recommend improvements based on secure signature standards.
3. An enterprise network uses Kerberos for authentication, but attackers successfully perform a replay attack to gain unauthorized access. Examine how this attack could occur and propose enhancements in the authentication process to prevent such threats.
4. A secure communication system uses a simple hash function to verify message integrity, but attackers are able to modify messages without detection. Analyze why this approach fails and propose a solution using HMAC to improve message authentication.
5. An organization implements X.509 certificates for secure communication, but users encounter frequent trust validation errors. Analyze the possible issues in certificate management and propose solutions to ensure proper authentication and trust chain validation.

RUBRICS FOR CALCULATION:

Q. No	Understanding of Concepts (25)	Experimenta l Design (30)	Use of Tools (20)	Analysis of Results (15)	Documenta tion (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

R. Deepasri (192511309)

CO3 AT2 CRYPTOGRAPHY:

CASE STUDY

1. Case Study: MD5 Password Hashing

Problem

A company stores user passwords using the **MD5 hashing algorithm**. After a security breach, attackers obtained the password database and were able to crack a large number of user passwords.

Explanation

MD5 stands for **Message Digest Algorithm 5**. It is a cryptographic hash function that converts input data of any size into a fixed **128-bit message digest**.

A basic hashing process can be represented as:

Password → MD5 Hash → Stored in Database

For example, instead of storing the actual password, the system stores its hash value. When a user logs in, the entered password is hashed and compared with the stored hash.

However, **MD5 is not considered secure for password storage**.

Why MD5 is insecure

1. MD5 is very fast:

A password-hashing algorithm should be deliberately slow so that attackers cannot perform a huge number of guesses quickly. MD5 was designed to be fast, which makes brute-force and dictionary attacks much easier.

2. Collision vulnerabilities:

MD5 has serious cryptographic weaknesses that allow attackers to construct different inputs with the same hash under certain conditions. Therefore, MD5 should not be used for modern security-sensitive applications.

3. Rainbow-table attacks:

If passwords are stored as unsalted MD5 hashes, attackers can use precomputed tables containing common passwords and their hashes.

4. Weak passwords can be easily guessed:

If a user chooses a common password such as a simple word or short password, an attacker can calculate its MD5 hash and compare it with the stolen database.

5. No protection against offline attacks:

Once an attacker obtains the password database, they can perform password guesses without interacting with the company's login system.

Recommended solution

The company should replace MD5 with a modern **password hashing algorithm**, such as:

- **Argon2id**
- **bcrypt**
- **scrypt**
- **PBKDF2**

A unique random **salt** should be generated for every user.

The process becomes:

Password + Unique Salt → Password Hashing Algorithm → Stored Hash

The salt does not have to be secret. Its purpose is to make identical passwords produce different stored hashes and to make precomputed attacks much less useful.

The system should also enforce strong password policies and, where appropriate, use **multi-factor authentication (MFA)**.

Conclusion

MD5 is unsuitable for modern password storage because it is fast and has known cryptographic weaknesses. The company should migrate to a **slow, salted, password-specific hashing algorithm such as Argon2id**, while also protecting accounts with additional authentication controls.

2. Case Study: Digital Signature and Unauthorized Transactions

Problem

A banking application uses **digital signatures** to authorize transactions. However, unauthorized transactions are still being approved. The organization needs to determine why the digital-signature system is failing and how it can be improved.

Explanation

A digital signature is a cryptographic mechanism used to verify:

- **Authentication** – confirms who signed the data.
- **Integrity** – ensures the signed data has not been modified.
- **Non-repudiation** – provides evidence that the holder of the signing key authorized the data, assuming the private key was properly controlled.

Digital signatures normally use a **private key** for signing and a corresponding **public key** for verification.

The general process is:

Transaction → Hash → Sign with Private Key → Digital Signature

The receiver verifies the signature using the corresponding public key.

Possible reasons for unauthorized transactions

1. Private key compromise

The most serious possibility is that an attacker's access to a legitimate private key allows them to generate valid signatures.

Therefore, private keys must be strongly protected.

2. Incorrect transaction data being signed

The application may be signing only part of the transaction.

For example, if the signature covers only:

Transaction ID + Account Number

but does not cover:

Amount + Beneficiary

an attacker might modify the amount or beneficiary without invalidating the signature.

The complete security-sensitive transaction should be included in what is authenticated.

3. Replay attacks

An attacker may capture a previously valid signed transaction and attempt to submit it again.

For example:

Valid Transaction → Captured → Re-submitted

The system should use transaction IDs, timestamps, nonces or other freshness mechanisms to prevent this.

4. Weak private-key protection

If private keys are stored in insecure files or exposed to unauthorized applications, attackers may steal them.

5. Incorrect certificate validation

The banking application must verify that the public key belongs to the correct trusted entity. Improper certificate validation can result in accepting an untrusted signing key.

Security improvements

The organization should:

1. Protect private keys using secure key-management systems or **Hardware Security Modules (HSMs)** where appropriate.
2. Use modern, sufficiently strong signature algorithms.
3. Sign all critical transaction information.
4. Include unique transaction IDs and freshness information.
5. Verify the complete digital signature before approving transactions.
6. Correctly validate certificates and certificate chains.
7. Implement MFA or additional approval for high-value transactions.
8. Maintain detailed audit logs.
9. Monitor unusual transaction patterns.
10. Immediately revoke compromised certificates or keys.

Example

A transaction could contain:

Transaction ID + Sender + Receiver + Amount + Currency + Timestamp

The complete data should be authenticated so that changing any important field causes verification to fail.

Conclusion

Digital signatures themselves can provide strong security, but they are only effective when the **private keys, signed transaction data, certificates and verification process**

are properly protected. The banking application should ensure that every important transaction parameter is authenticated and that replay and key-compromise risks are addressed.

3. Case Study: Kerberos Replay Attack

Problem

An organization uses the **Kerberos authentication protocol** to provide secure authentication between users and network services. However, attackers are attempting replay attacks to gain unauthorized access.

Explanation

Kerberos is a network authentication protocol based on **secret-key cryptography** and a trusted third party.

It generally uses a **Key Distribution Center (KDC)**, which consists of:

- **Authentication Server (AS)**
- **Ticket Granting Server (TGS)**

A simplified process is:

User → Authentication Server → Ticket Granting Server → Service Server

The user receives tickets that can be used to authenticate to network services without repeatedly sending the user's password over the network.

What is a replay attack?

A replay attack occurs when an attacker captures a valid authentication message and attempts to use it again.

For example:

1. A legitimate user authenticates.
2. An attacker captures an authentication message.
3. The attacker stores the captured message.
4. The attacker later sends the same message to the server.

5. If the server does not check freshness correctly, it might accept the old message.

The attacker may therefore attempt to obtain unauthorized access without knowing the user's password.

How Kerberos prevents replay attacks

1. Timestamps

Kerberos authenticators contain timestamps. The server checks whether the authentication request was generated recently.

If a request is too old, it is rejected.

2. Clock synchronization

Because timestamps are important, systems using Kerberos must have reasonably synchronized clocks.

If computers have significantly different times, legitimate authentication can fail or security checks can become ineffective.

3. Session keys

Kerberos uses session keys to establish secure communication between authenticated parties.

4. Ticket lifetime

Tickets are valid only for a limited period. This reduces the usefulness of stolen or captured authentication information.

5. Authenticator checking

The server can check whether an authenticator is fresh and, depending on implementation, prevent reuse of the same authenticator.

Additional security measures

The organization should:

- Keep system clocks synchronized.
- Use secure time synchronization.
- Configure appropriate ticket lifetimes.
- Protect the KDC carefully.
- Protect Kerberos keys.
- Monitor authentication logs.

- Disable outdated or weak cryptographic configurations.
- Investigate repeated or unusual authentication attempts.
- Use strong account and administrative security controls.

Example

Suppose an authenticator has a timestamp of **10:00 AM** and the server receives it much later. If the timestamp is outside the permitted clock-skew window, the server rejects it rather than treating it as a new authentication request.

Conclusion

Kerberos protects against replay attacks primarily through **timestamps, authenticators, ticket lifetimes and session keys**. Proper clock synchronization and secure configuration are essential for Kerberos to work effectively.

4. Case Study: Simple Hash versus HMAC

Problem

An organization uses a normal cryptographic hash to verify the integrity of messages. However, attackers can modify messages and generate a new hash, making the system unable to determine whether the message came from a legitimate sender.

Explanation

A cryptographic hash function takes a message as input and produces a fixed-length output.

For example:

Message → SHA-256 → Hash Value

A hash is useful for detecting changes when the expected hash value is obtained through a trusted channel.

However, a normal hash function does **not contain a secret key**.

Therefore, if an attacker changes the message, the attacker can calculate a new hash for the modified message.

Example of the problem

Suppose the original message is:

"Transfer ₹5,000"

The system calculates:

Message → Hash A

An attacker changes it to:

"Transfer ₹50,000"

The attacker can calculate:

Modified Message → Hash B

If the receiver receives both the modified message and the attacker's newly calculated hash, a simple hash does not prove that the message came from the legitimate sender.

HMAC solution

The organization should use **HMAC (Hash-based Message Authentication Code)**.

HMAC combines:

- A secret key
- The message
- A cryptographic hash function

Conceptually:

Secret Key + Message → HMAC Algorithm → Authentication Code

The sender and receiver share a secret key.

The sender calculates the HMAC and sends:

Message + HMAC

The receiver calculates the HMAC again using the same secret key.

If the calculated HMAC matches the received HMAC, the message is considered authentic and unchanged, assuming the key is secure.

Why HMAC is better

1. Integrity

HMAC detects whether the message has been modified.

2. Authentication

It provides assurance that the message was generated by someone possessing the shared secret key.

3. Protection against modification

An attacker who does not know the secret key cannot normally create a valid HMAC for modified content.

4. Efficient

HMAC is efficient and widely used in security protocols and applications.

Example

Original:

Message + Secret Key → HMAC-SHA-256 → HMAC

If an attacker changes the message:

Modified Message + Secret Key → Different HMAC

Because the attacker does not know the secret key, they cannot normally generate the correct new HMAC.

Important difference

Normal Hash	HMAC
Uses a hash function	Uses a secret key with a hash
Detects changes when the expected hash is trusted	Provides integrity and authentication
No secret key	Requires a shared secret key
Attacker can calculate a new hash	Attacker cannot normally calculate a valid HMAC without the key
Example: SHA-256	Example: HMAC-SHA-256

Conclusion

A simple hash alone is insufficient when **message authentication** is required. The organization should use **HMAC-SHA-256 or another secure HMAC construction** to provide both message integrity and authentication.

5. Case Study: X.509 Certificate Trust Validation

Problem

An organization uses **X.509 digital certificates** to secure communication and authenticate servers. However, users frequently receive certificate trust-validation errors.

Explanation

An **X.509 certificate** is a digital certificate used to associate an identity with a public key.

Certificates are normally issued and digitally signed by a **Certificate Authority (CA)**.

A typical certificate trust hierarchy is:

Root CA

↓

Intermediate CA

↓

Server/End-Entity Certificate

The client verifies the certificate and builds a chain from the server certificate to a trusted root CA.

Common causes of certificate errors

1. Expired certificate

Every certificate has a validity period.

If the certificate has passed its expiration date, the client should reject it.

2. Incorrect system date and time

Certificate validation depends on time.

If a computer's date is incorrect, a perfectly valid certificate may appear expired or not yet valid.

3. Hostname mismatch

The domain name used by the client must match the identity listed in the certificate, normally through the **Subject Alternative Name (SAN)** extension.

For example, a certificate issued for:

example.com

should not automatically be trusted for an unrelated hostname.

4. Missing intermediate certificate

The server may provide its own certificate but fail to provide the required intermediate CA certificate.

This can prevent the client from building the complete trust chain.

5. Untrusted root CA

If the root CA is not present in the client's trusted certificate store, the certificate chain may not be trusted.

6. Revoked certificate

A certificate can be revoked before its normal expiration date if its private key is compromised or other security problems occur.

7. Incorrect server configuration

The wrong certificate may be installed, or the certificate may not correspond to the private key used by the server.

Solutions

The organization should:

1. Install a valid certificate from a trusted CA.
2. Configure the complete certificate chain.
3. Install the necessary intermediate certificates.
4. Ensure trusted root certificates are correctly configured.
5. Keep server and client clocks synchronized.
6. Ensure the certificate's SAN matches the hostname.
7. Monitor certificate expiration dates.
8. Renew certificates before expiration.
9. Check certificate revocation status where appropriate.
10. Protect the certificate's private key.
11. Regularly audit TLS/certificate configurations.
12. Automate certificate renewal where practical.

Example

Suppose a user visits:

bank.

example

The server sends its certificate.

The browser checks:

Server Certificate → Intermediate CA → Trusted Root CA

It also checks the validity period, hostname, signature and other applicable certificate properties.

If the chain ends at a trusted root and the other checks succeed, the certificate can be trusted. If the chain cannot be validated, the browser may display a certificate warning.

Conclusion

X.509 certificate validation depends on a correctly configured **trust chain, valid certificate, trusted CA, correct hostname, accurate system time and appropriate revocation checking**. Proper certificate lifecycle management is therefore essential for secure communication.
